

AstraPIO

Coproc sseur programmable pour entr es sorties num riques

Datasheet et manuel de r f rence logiciel

ABI 5 • Moteur temporel 0x0118 • R vision documentaire 1.0 • 20 septembre 2026

AstraPIO d charge un microcontr leur h te des s quences num riques d terministes. Un interpr teur PIO ex cute un programme court, tandis qu'un moteur temporel ind pendant capture, remplace et r g n re des impulsions. L'h te charge les param tres et  change des donn es par SPI ; il ne cadence pas les bits du flux temporel.

Caract ristiques principales

Fonction	Caract�ristique
PIO	Un contexte ; accumulateur 8 bits ; compteur de boucle 4 bits
Programme	16 instructions de 10 bits ; 160 bits utiles en latches
Ex�cution	Un cr�neau toutes les 4 p�riodes CLK ; 80 ns � 50 MHz
Donn�es	FIFO TX de 2 octets et FIFO RX de 2 octets
Moteur temporel	Pr�fixe de 1 � 24 bits ; relais et remplacement atomique
Interface h�te	SPI esclave mode 0 ; commande 8 + adresse 8 + donn�e 16 bits
Broches applicatives	13 sources d'entr�e et 14 destinations de sortie logiques ; 8 partag�es
Int�gration	Tiny Tapeout IHP SG13G2 ; allocation 1 � 2 tiles
Identit�s	PIO_ID = 0x5049 ; ABI = 0x0500 ; TIMED_ID = 0x5449

Applications

G n ration de s quences GPIO,  mission UART ou SPI par programme, capture de signaux   largeur d'impulsion, interfaces temporelles personnalis es. Le remplacement d'un pr fixe WS2812 avec relais de la suite est une application valid e en simulation, et non une fonction câbl e sp cifique aux LED.

Statut du produit

Donn es pr liminaires avant qualification sur silicium. Le design a pass  les contr les physiques et a  t  accept  dans le shuttle TTIHP26b. Les performances sur carte, caract ristiques  lectriques, tol rances et limites absolues ne constituent pas encore des valeurs garanties. Ce document d crit le circuit exact soumis, sans les extensions exp rimentales ABI 6.

R f rence circuit 1b1c91183a4a9a5ea3516699845336175ffe6d96
Projet Tiny Tapeout 5799 • PR149 • Bloc RTL tt_um_fabien_pio

2 Guide de lecture

Ce document s'adresse aux concepteurs de cartes, développeurs de pilotes et auteurs de programmes PIO. Il réunit la description fonctionnelle du bloc et son interface de programmation. Les noms de registres utilisés ici sont des noms documentaires ; les adresses et champs proviennent du RTL ABI 5.

Section	Contenu
3 à 7	Architecture, brochage, GPIO, reset et caractéristiques électriques
8 et 9	Format SPI et contraintes temporelles du transport
10 à 12	Registres PIO, FIFO et mémoire programme
13 et 14	Jeu d'instructions ABI 5 et règles d'exécution
15 à 18	Moteur temporel, registres et chronogrammes
19 et 20	Connexion LPC546xx et application WS2812
21 et 22	Exemples PIO et API portable C
23 et 24	Erreurs, limites, qualification et références

Conventions

Notation	Signification
0x / [m:n]	Valeur hexadécimale / champ allant du bit m au bit n, inclus
R / W / R/W	Lecture / écriture / lecture et écriture
W1C / strobe	Écrire 1 pour acquitter / commande ponctuelle non mémorisée
Reset	Valeur observable après reset global effectif, sauf mention contraire
TCLK	Période du CLK du bloc ; 20 ns dans la configuration à 50 MHz
TX / RX	Point de vue du PIO : TX hôte → PIO ; RX PIO → hôte
Donnée non valide	Le registre retourne zéro lorsque cette règle est explicitement indiquée

Portée des valeurs

« Fonctionnel » décrit les règles implémentées. « Calculé » désigne une conséquence arithmétique du RTL. « Simulé » concerne les bancs de test ; « STA » l'analyse temporelle statique. Aucun de ces termes ne signifie caractérisation électrique sur des puces fabriquées.

Les bits réservés se lisent à zéro dans les registres implémentés ; écrire zéro aux bits réservés, sauf champ explicitement décrit comme tronqué. Les adresses non implémentées se lisent à zéro mais les écritures provoquent généralement HOST_ERROR. Les anciens binaires ABI 3 et ABI 4 ne sont pas compatibles avec ABI 5.

3 Architecture fonctionnelle

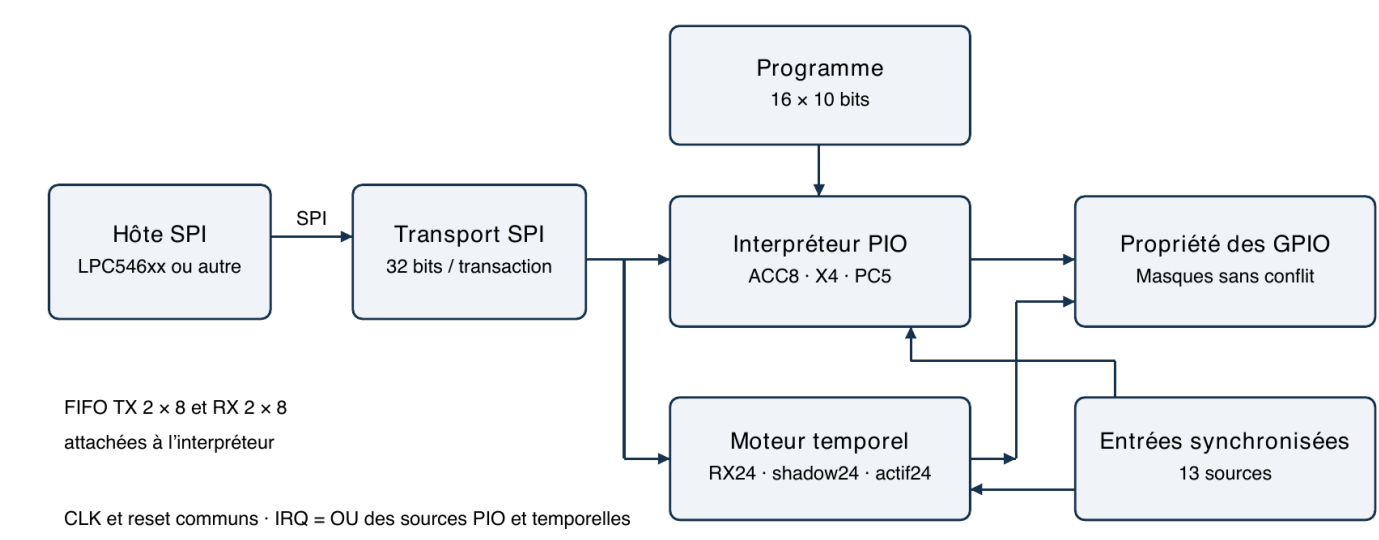


Figure 1 Organisation logique du bloc AstraPIO

Le PIO et le moteur temporel fonctionnent simultan ment. Il n'existe ni ordonnanceur multicontexte ni second interpr teur. La phase d'ex cution du PIO tourne en quatre cycles ; le moteur temporel travaille   chaque cycle CLK.

Ressource	Capacit�	Usage
M�moire programme	16 x 10 bits	Instructions uniquement ; pas une RAM de donn�es g�n�rale
FIFO PIO	TX 2 x 8 ; RX 2 x 8	Flux d'octets avec blocage de PULL/PUSH
�tat PIO	ACC 8 ; X 4 ; PC 5 bits	Calcul, boucle et position dans le programme
Banques temporelles	3 x 24 bits	Capture RX, staging TX et mot actif TX
Broches temporelles	1 entr�e ; 0 ou 1 sortie	S�lection parmi les broches logiques disponibles

Aucune SRAM macro, DMA int gr , m moire non volatile, convertisseur analogique ou PLL n'est pr sent dans AstraPIO. Le DMA  ventuel appartient au LPC. La sortie du moteur temporel ne traverse pas les FIFO PIO.

4 Brochage logique

Les noms ci-dessous d signent les ports du projet Tiny Tapeout. Ils ne sont pas des num ros de bo tier ni de connecteur. Le brochage physique d pend du shuttle et de la carte porteuse. Les alimentations et la s lection de projet rel vent de cette infrastructure.

Port	Sens	Fonction AstraPIO	Reset du bloc
clk	Entr�e	Horloge principale externe	Sans objet
rst_n	Entr�e	Reset global actif � z�ro	Actif � 0
ena	Interne TT	S�lection / activation du bloc	0 force le reset interne
ui_in[0]	Entr�e	HOST_SCK ; horloge SPI h�te	SCK attendu � 0
ui_in[1]	Entr�e	HOST_MOSI ; donn�es SPI h�te	Indiff�rent hors transfert
ui_in[2]	Entr�e	HOST_CS_N ; s�lection SPI	CS_N attendu � 1
ui_in[3]	Entr�e	Source PIO I8	�chantillonn�e apr�s reset
ui_in[4]	Entr�e	Source PIO I9	Idem
ui_in[5]	Entr�e	Source PIO I10	Idem
ui_in[6]	Entr�e	Source PIO I11	Idem
ui_in[7]	Entr�e	Source PIO I12	Idem
uo_out[0]	Sortie	HOST_MISO ; sortie SPI push-pull	0
uo_out[1]	Sortie	HOST_IRQ ; interruption active � 1	0
uo_out[2]	Sortie	Destination PIO O8	0
uo_out[3]	Sortie	Destination PIO O9	0
uo_out[4]	Sortie	Destination PIO O10	0
uo_out[5]	Sortie	Destination PIO O11	0
uo_out[6]	Sortie	Destination PIO O12	0
uo_out[7]	Sortie	Destination PIO O13	0
uio[7:0]	Bidir.	Sources I7:I0 et destinations O7:O0	Sorties d�sactiv�es

R gles de c blage

Les six sorties uo_out[7:2] ne deviennent pas haute imp dance quand le programme s'arr te. MISO ne devient pas haute imp dance quand CS_N est inactif. Pr voir un bus h te d di  ou un isolement externe de MISO ; ne pas relier plusieurs sorties push-pull directement.

Le projet 5799 est un identifiant de plateforme, pas un num ro de s lection mat rielle   coder en dur. Utiliser la correspondance de la carte et du shuttle effectivement livr s.

5 GPIO et propri t  des broches

Broche	Index entr�e	Index sortie	Commande de direction
uio0	0	0	DIR, bit ACC0 ; ou moteur temporel
uio1	1	1	DIR, bit ACC1 ; ou moteur temporel
uio2	2	2	DIR, bit ACC2 ; ou moteur temporel
uio3	3	3	DIR, bit ACC3 ; ou moteur temporel
uio4	4	4	DIR, bit ACC4 ; ou moteur temporel
uio5	5	5	DIR, bit ACC5 ; ou moteur temporel
uio6	6	6	DIR, bit ACC6 ; ou moteur temporel
uio7	7	7	DIR, bit ACC7 ; ou moteur temporel
ui_in[3:7]	8 � 12	Aucun	Entr�es d�di�es
uo_out[2:7]	Aucun	8 � 13	Sorties d�di�es, toujours pilot�es

Les 13 entr es et 14 sorties logiques ne repr sentent pas 27 broches distinctes : huit sont bidirectionnelles. L'interface h te utilise trois entr es, deux sorties et les ports communs de contr le.

Masque du PIO

Le registre OUTPUT_MASK   0x10 r serve un ensemble de sorties O0   O13, m me lorsque RUN vaut z ro. Il se modifie uniquement PIO arr t . Aucun bit r serv  au moteur temporel ne peut  tre revendiqu . Une  criture accept e met   z ro les anciens niveaux et directions poss d s par le PIO avant d'installer le nouveau masque.

SET, OUT et OUTBIT ne modifient que les sorties poss d es. Une destination valide mais non poss d e n'est pas une erreur. OUTBIT d cale n anmoins l'accumulateur. DIR agit seulement sur les huit sorties uio poss d es ; un bit   1 active leur driver, un bit   0 le d sactive.

Moteur temporel

Lorsque ENABLE et OUTPUT_ENABLE valent 1, le moteur r serve sa destination DOUT et force son output-enable si elle appartient   uio. Le multiplexeur s lectionne alors sa valeur. Les conflits de propri t  sont rejet s par HOST_ERROR ; ils ne sont pas arbitr s dynamiquement.

 chantillonnage et lecture

Les entr es traversent deux registres de synchronisation. INPUT_SAMPLE (0x09) donne ces valeurs synchronis es. Ce chemin ne remplace pas une garantie de largeur minimale d'impulsion ni un filtre anti-rebond. Les registres 0x07 et 0x08 lisent les  tats du PIO, pas la sortie finale ni la direction impos e par le moteur temporel.

6 Horloge reset et démarrage

Horloge

Le domaine logique unique est CLK. SCK est échantillonné par ce domaine ; ce n'est pas une seconde horloge interne. La configuration soumise vise 50 MHz, soit $TCLK = 20$ ns. Les délais PIO et temporels sont des nombres de cycles ; changer CLK modifie toutes leurs durées.

Événement	Effet
$rst_n = 0$ ou $ena = 0$	Assertion asynchrone du reset interne et masquage des sorties ; $uio_oe = 0$
$rst_n = 1$ et $ena = 1$	Libération interne synchronisée par deux bascules ; horloge nécessaire
Reset global	Programme invalidé, $RUN = 0$, FIFO vides, masques et erreurs effacés ; moteur temporel désactivé
$RUN = 0$	Interpréteur arrêté ; état, données et niveaux GPIO conservés
$RESTART = 1$	Interpréteur arrêté, PC/ACC/X/délai/IRQ/événement remis à zéro ; FIFO vidées ; défaut PIO effacé
Arrêt temporel	Trame abandonnée et réservation DOUT relâchée ; ne remet pas toutes les banques et erreurs à zéro

RESTART ne supprime ni le programme ni OUTPUT_MASK ni les niveaux/directions GPIO. Il n'efface pas le HOST_ERROR du PIO et ne réinitialise pas le moteur temporel. Les données physiques non resetées des latches sont masquées par les longueurs et indicateurs de validité.

Séquence conseillée de mise en service

- Sélectionner AstraPIO sur la carte ; empêcher les autres circuits de piloter ses entrées hôte et ses sorties uio.
- Fournir une horloge stable. Maintenir $CS_N = 1$ et $SCK = 0$ pendant le reset.
- Maintenir le reset au moins cinq cycles CLK, puis attendre au moins cinq cycles après sa libération avant la première transaction. Cette séquence reproduit les tests, sans constituer une limite électrique mesurée.
- Lire PIO_ID, ABI, CONTEXTS et CAPACITY ; attendre respectivement 0x5049, 0x0500, 0x0001 et 0x1002.
- Configurer et charger les ressources arrêtées, relire la configuration, vérifier les erreurs, puis démarrer.

AstraPIO ne possède pas de démarrage autonome en ROM : le programme et les paramètres applicatifs doivent être rechargés après un reset global. Arrêter CLK pendant une activité fige l'état et peut maintenir une sortie haute ; aucune mise en sécurité automatique par watchdog n'est intégrée.

7 Caractéristiques électriques et temporelles

Les caractéristiques ci-dessous séparent le contrat logique du bloc des paramètres de la puce complète. Les valeurs de coins STA ne constituent pas une plage de fonctionnement électrique garantie.

Paramètre	Valeur ou statut	Qualification
Technologie	IHP SG13G2	PDK gelé, voir section 24
Tension cœur nominale du modèle	1,20 V	Coin de bibliothèque, pas une alimentation de connecteur
Coins de cellules audités	1,32 V / -40 °C ; 1,20 V / 25 °C ; 1,08 V / 125 °C	STA avec RC nominal extrait
Horloge de référence	50 MHz / 20 ns	Contraintes et simulations du design soumis
Alimentation IO de la carte	À confirmer sur la carte TTIHP26b livrée	Ne pas déduire VDDIO de la tension cœur
VIH, VIL, VOH, VOL, IOH, IOL	Non spécifiés pour AstraPIO	Caractérisation de l’infrastructure / carte requise
Courant actif et repos	Non caractérisés	Pas de budget de puissance garanti
Capacité et temps de montée des IO	Non caractérisés sur carte	Dépendent des pads, multiplexeurs et charges
Limites absolues et ESD	Non spécifiées	Aucune tolérance 5 V ou protection garantie
Température de fonctionnement	Non qualifiée sur silicium	Ne pas convertir les coins STA en grade commercial

Dimensions du bloc

La boîte de placement finale mesure 202,08 × 313,74 µm, soit 0,0634006 mm². Il s’agit du bloc utilisateur 1 × 2 tiles, pas du die complet ni d’un boîtier. La surface de cellules est 56 841,5 µm², soit 94,5637 % du cœur utilisable.

Intégration électrique

Relier les masses, vérifier les rails de la carte, les seuils et la tolérance de chaque broche du LPC546xx choisi. Un signal WS2812 provenant d’un domaine 5 V ne doit pas être appliqué directement sans vérification ; prévoir les adaptations de niveau nécessaires. Une sortie logique AstraPIO ne fournit pas le courant d’alimentation des LED.

Les pages GPIO/Clock génériques de Tiny Tapeout décrivent notamment des pads SKY130. Leurs valeurs de courant, de tension et de fréquence ne sont pas transposées au bloc IHP. Le schéma de la carte TTIHP26b effectivement livrée reste la référence pour le raccordement physique. [R7]

8 Protocole SPI hôte

AstraPIO est un esclave SPI en mode 0 (CPOL = 0, CPHA = 0), MSB en premier. Une assertion de CS_N transporte exactement quatre octets. L'adresse est celle d'un registre de 16 bits ; elle n'est pas incrémentée automatiquement.

Octet	MOSI écriture	MOSI lecture	MISO lecture
0	0x02	0x03	Ignorer
1	Adresse	Adresse	Ignorer
2	DATA[15:8]	Dummy, généralement 0	DATA[15:8]
3	DATA[7:0]	Dummy, généralement 0	DATA[7:0]

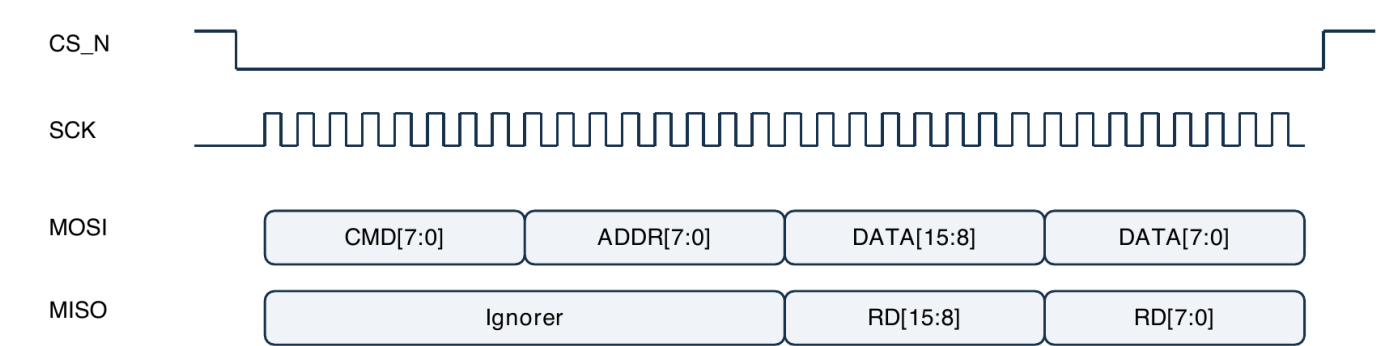


Figure 2 Découpage de la transaction SPI ; chronogramme logique non à l'échelle

Exemples de trames

Lecture ID	MOSI 03 00 00 00	MISO xx xx 50 49
Lecture ABI	MOSI 03 01 00 00	MISO xx xx 05 00
Arrêt PIO	MOSI 02 03 00 00	
Chargement HALT	MOSI 02 40 03 06	mot ABI5 = 0x306

Les écritures prennent effet lorsque le dernier bit a été reçu et transféré dans le domaine CLK, sans attendre une nouvelle transaction. Une écriture interrompue avant 32 bits n'est pas appliquée. Les bits supplémentaires sous la même assertion de CS_N sont ignorés : ce n'est pas un mode burst.

Une commande différente de 0x02/0x03 est ignorée et n'émet pas d'acquittement. Il n'y a ni CRC, ni bit ACK sur le fil, ni notification fiable de violation des timings SPI. PIO_OK dans le pilote signifie d'abord succès du transport ; relire l'état ou la valeur pour vérifier l'acceptation matérielle.

9 Contraintes SPI et atomicité

Symbole	Contrainte logique	À 50 MHz
tSCKH	SCK haut ≥ 6 TCLK	≥ 120 ns
tSCKL	SCK bas ≥ 6 TCLK	≥ 120 ns
tCSS	CS_N bas avant le premier front actif ≥ 6 TCLK	≥ 120 ns
tCSH	CS_N maintenu bas après la fin de l'horloge ≥ 6 TCLK	≥ 120 ns
tCSG	CS_N haut entre deux transactions ≥ 6 TCLK	≥ 120 ns
fSCK	$\leq fCLK / 12$, sous réserve de toutes les autres contraintes	$\leq 4,1667$ MHz calculés
MOSI	Changer au front descendant et conserver jusqu'au front montant suivant	Mode 0 ; pas de valeur pad setup/hold garantie

La règle de six cycles provient du contrat RTL/driver et des simulations avec phases d'horloge balayées. Elle ne remplace pas une caractérisation des délais des pads de la puce complète. Pour le premier essai matériel, choisir une cadence plus lente, puis mesurer les marges.

Instantané de lecture

Le registre est échantillonné au front SCK descendant synchronisé qui suit la réception du seizième bit (fin d'adresse). Les seize bits suivants sortent de cet instantané, même si le registre interne change ensuite. Les bits MOSI de cette phase de lecture sont ignorés.

Lecture destructive de la FIFO RX

À 0x15, le bit 15 de l'instantané indique si un octet était présent. Un retrait FIFO n'a lieu qu'à la réception complète des 32 bits et seulement si cet instantané était valide. Une lecture abandonnée ne consomme rien. Une lecture initialement vide ne consomme pas un octet arrivé plus tard.

Après une erreur de transport dont l'instant exact est inconnu, ne pas relancer aveuglément une lecture RX : la première transaction peut déjà avoir consommé l'octet. La même incertitude existe pour une écriture ou un COMMIT dont le dernier bit a peut-être atteint le circuit.

Débit et service hôte

Une transaction utilise 32 clocks SCK pour au plus 16 bits de registre, ou un octet de FIFO. À 4,1667 MHz, la limite brute est 130 208 transactions/s avant les gaps CS et les lectures de statut. Ce chiffre calculé n'est pas un débit garanti du LPC ou des protocoles applicatifs. Les deux octets de FIFO ne permettent pas de cacher une latence hôte arbitraire.

10 Registres globaux du PIO

Toutes les adresses sont hexadécimales. Les valeurs de reset sont observables après reset global. Les bits non nommés sont réservés. Le contexte unique utilise le bit 0 des masques de commande.

Adr.	Nom	Accès	Reset	Champs et effet
00	PIO_ID	R	5049	Identité du bloc
01	ABI	R	0500	Version de l'interface et de l'encodage
02	CONTEXTS	R	0001	Un interpréteur
03	RUN	R/W	0000	[0] exécution. W : 0 arrêt ; 1 marche si pas de défaut et mémoire libre ; autre valeur rejetée
04	RESTART	W	0000	W=1 : restart et vidage FIFO ; W=0 sans effet ; lecture 0
05	PIO_IRQ	R/W1C	0000	[0] IRQ logicielle. W=1 efface cette seule source ; W=0 conserve
06	PIO_ERROR	R/W	0000	R : [8] FAULT, [0] HOST_ERROR. W=1 efface HOST_ERROR uniquement ; autre valeur invalide
07	PIO_OUT	R	0000	[13:0] niveaux mémorisés par le PIO, avant le multiplexeur temporel
08	PIO_OE	R	0000	[7:0] directions uio du PIO, avant réservation temporelle
09	INPUT_SAMPLE	R	0000	[12:0] entrées synchronisées ; évoluent après reset
0A	PROGRAM_LENGTH	R/W	0000	R : [4:0] longueur 0..16. W=0 arrêté et non busy : invalide le programme et PC=0
0B	PROGRAM_BUSY	R	0000	[0] écriture mémoire interne en cours
0C	RX_IRQ_MASK	R/W	0000	[0] active l'IRQ tant que la FIFO RX est non vide
0D	EVENT_SET	R/W	0000	R : [0] événement. W=1 le pose ; W=0 sans effet
0E	EVENT_CLEAR	R/W	0000	R : [0] événement. W=1 l'efface ; W=0 sans effet
0F	CAPACITY	R	1002	[15:8]=16 mots ; [7:0]=2 octets par FIFO

Aux adresses 0x05, 0x0C, 0x0D et 0x0E, seules les valeurs 0 et 1 sont acceptées. Les écritures sur les registres R sont rejetées par HOST_ERROR. RUN n'effectue pas de restart implicite ; HALT et STOP conservent les sorties.

Interruption partagée

HOST_IRQ = PIO_IRQ OR FAULT OR HOST_ERROR OR (RX_IRQ_MASK AND RX non vide) OR IRQ temporelle. Il s'agit d'un niveau actif haut, pas d'une impulsion. Aucun masque global ne masque FAULT, HOST_ERROR ou les sources temporelles. Lire les deux pages d'état avant d'acquitter.

11 Registres de contexte et FIFO

Adr.	Nom	Acc�s	Reset	Description
10	OUTPUT_MASK	R/W	0000	[13:0] sorties poss�d�es ; modification arr�t� ; aucun conflit temporel
11	PC	R/W	0000	R : [4:0] PC courant. W : adresse 0..15, arr�t� uniquement
12	ACC	R/W	0000	[7:0] accumulateur ; W arr�t� ; bits hauts de W ignor�s
13	DELAY_LEFT	R	0000	[7:0] cr�neaux de d�lai restants
14	TX_DATA	W	0000	W : ajoute les 8 bits bas � TX ; bits hauts ignor�s. Lecture 0
15	RX_DATA	R pop	0000	[15] VALID ; [7:0] octet. Lecture compl�te valide consomme un octet
16	FIFO_STATUS	R	0005	Niveaux et drapeaux ci-dessous
17	LOOP_COUNTER	R/W	0000	[3:0] compteur X ; W arr�t� ; bits hauts ignor�s
1F	PROGRAM_LENGTH_ALIAS	R	0000	[4:0] longueur programme ; alias de lecture de 0x0A

Bits de 0x16	Nom	Signification
[15:14]	RX_LEVEL	Nombre d'octets RX, de 0 � 2
[13:12]	TX_LEVEL	Nombre d'octets TX, de 0 � 2
[11:4]	R�serv�s	Z�ro
[3] / [2]	RX_FULL / RX_EMPTY	RX pleine / vide
[1] / [0]	TX_FULL / TX_EMPTY	TX pleine / vide

R gles de transfert

TX_DATA peut  tre  crit PIO arr t  ou actif. Une  criture alors que TX est pleine est rejet e avec HOST_ERROR, sauf si un PULL effectif lib re simultan ment une place. PULL bloque le programme quand TX est vide ; PUSH bloque quand RX est pleine, sans effacer de donn e.

Il n'existe pas de bypass combinatoire d'une FIFO vide. Une arriv e et un retrait simultan s conservent le niveau quand tous deux sont accept s. RESTART et reset invalident les deux FIFO. Ni RUN=0 ni HALT ne les vident.

Lire 0x15 vide renvoie 0 et ne provoque pas d'erreur. Lire 0x16 n'est pas destructif. Les adresses 0x18..0x1E et 0x20..0x2F ne sont pas un second contexte : lecture 0,  criture invalide.

12 Mémoire programme et lancement

Les adresses 0x40..0x4F contiennent chacune une instruction ABI 5 dans les bits [9:0] d'une transaction de 16 bits. Les bits [15:10] doivent être nuls. Le stockage utile de 160 bits n'est pas une RAM de 20 octets adressable octet par octet.

Opération	Condition et résultat
Écriture de programme	RUN=0, BUSY=0, mot ≤ 0x03FF et adresse locale ≤ longueur actuelle
Ajout d'une instruction	Adresse locale = longueur : la longueur est incrémentée
Remplacement d'une instruction	Adresse locale < longueur : la longueur est conservée
Trou dans le programme	Adresse locale > longueur : rejet, HOST_ERROR ; aucun allongement
Lecture	Donnée valide seulement si arrêté, non busy et index < longueur ; sinon zéro
Opcode réservé	Peut être stocké et relu ; déclenche FAULT s'il est exécuté
Fin du programme	PC ne reboucle pas de 15 vers 0 ; un accès PC ≥ longueur arrête avec FAULT

Séquence de chargement sûre

```
W(0x03, 0x0000);      // arrêter le PIO
W(0x04, 0x0001);      // restart et purge des FIFO
W(0x06, 0x0001);      // effacer HOST_ERROR si nécessaire
attendre R(0x0B) == 0;
W(0x0A, 0x0000);      // invalider le programme
for (i = 0; i < N; ++i) {
    W(0x40 + i, code10[i]);
    attendre R(0x0B) == 0;
    vérifier R(0x40 + i) == code10[i];
}
W(0x10, masque);      // propriété des sorties
W(0x11, entree);       // 0 ≤ entree < N
vérifier R(0x0A) == N et R(0x06) == 0;
W(0x03, 0x0001);
```

W/R représentent ici une transaction SPI complète, avec les timings requis et contrôle de l'erreur de transport. Le polling doit avoir un timeout logiciel. N doit être entre 1 et 16. Relire OUTPUT_MASK et RUN permet de vérifier leur acceptation.

L'écriture interne des latches occupe deux cycles après la prise de requête. Le rythme SPI conforme est nettement plus lent, mais BUSY reste l'indication de référence. Les mots ne sont pas physiquement effacés au reset : PROGRAM_LENGTH=0 les rend inaccessibles.

Assemblage

```
python3 tools/pioasm.py --abi 5 examples/compact/uart_tx.pio
```

Toujours préciser --abi 5 : le défaut de l'outil est ABI 3 pour compatibilité historique. Une ligne d'instruction occupe un mot ; les labels et commentaires ne consomment pas de mémoire.

13 Instructions et calcul

ACC est l'accumulateur de 8 bits, X le compteur de 4 bits et PC le pointeur d'instruction. Une instruction s'exécute dans un créneau de quatre cycles CLK. Hors branchement/blocage, PC est incrémenté. Les opérations arithmétiques bouclent à leur largeur, sans registre de drapeaux général.

Instruction	Code 10 bits	Effet
LDI k	0x000 + k	$ACC \leftarrow k$, k de 0 à 255
DELAY k	0x100 + k	Charge k créneaux d'attente après cette instruction
XOR k	0x200 + k	$ACC \leftarrow ACC \text{ XOR } k$
NOP	0x300	Aucun effet autre que PC
DIR	0x301	Directions uio possédées $\leftarrow ACC[7:0]$
DEC	0x302	$ACC \leftarrow ACC - 1$ modulo 256
SHL	0x303	$ACC \leftarrow \{ACC[6:0], 0\}$
SHR	0x304	$ACC \leftarrow \{0, ACC[7:1]\}$
IRQ	0x305	Pose PIO_IRQ ; ne bloque pas le programme
HALT	0x306	$RUN \leftarrow 0$; PC avance ; sorties conservées
PULL	0x307	$ACC \leftarrow$ prochain octet TX ; bloque si vide
PUSH	0x308	Ajoute ACC à RX ; bloque si pleine
AWAIT	0x309	Attend EVENT puis le consomme
CLR_EVENT	0x30A	Efface EVENT
IN	0x30B	$ACC \leftarrow$ entrées synchronisées I7:I0
IN 1	0x30C	$ACC[4:0] \leftarrow I12:I8$; $ACC[7:5] \leftarrow 0$
OUT	0x30D	Sorties possédées O6:O0 $\leftarrow ACC[6:0]$
OUT 1	0x30E	Sorties possédées O13:O7 $\leftarrow ACC[6:0]$

Durée et blocage

À 50 MHz, les créneaux valent 80 ns, soit au plus 12,5 millions d'instructions/s calculés. DELAY k ajoute k créneaux vides : l'instruction suivante est exécutée $(k+1) \times 4$ TCLK après DELAY. WAIT, PULL, PUSH et AWAIT retentent à chaque créneau, pas à chaque cycle CLK.

Une instruction HALT n'est pas un reset. Relancer RUN reprend au PC suivant ; si ce PC dépasse la longueur, FAULT est levé. Il n'existe pas d'instruction ADD, de pile, d'appel de sous-programme ou de multiplication.

14 Branchements et instructions de broche

Instruction	Code	Effet et bornes
JMP a	0x310 a	$PC \leftarrow a$; a de 0 à 15
JNZ a	0x320 a	$PC \leftarrow a$ si $ACC \neq 0$
JBIT a	0x330 a	$PC \leftarrow a$ si $ACC[7] = 1$
WAIT p,0	0x340 p	Attend que l'entrée Ip synchronisée vaille 0 ; p de 0 à 12
WAIT p,1	0x350 p	Attend que l'entrée Ip synchronisée vaille 1 ; p de 0 à 12
SET p,0	0x360 p	Met Op à 0 si possédée ; p de 0 à 13
SET p,1	0x370 p	Met Op à 1 si possédée ; p de 0 à 13
OUTBIT p	0x380 p	$Op \leftarrow ACC[7]$ si possédée, puis $ACC \leftarrow ACC \ll 1$; p de 0 à 13
INBIT p	0x390 p	$ACC \leftarrow \{ACC[6:0], Ip\}$; p de 0 à 12
LDX n	0x3A0 n	$X \leftarrow n$; n de 0 à 15
DJNZ a	0x3B0 a	$X \leftarrow X - 1$ modulo 16 ; branche si ancien $X \neq 1$

Cas particuliers

LDX 0 suivi d'une boucle DJNZ réalise seize passages avant sortie, par bouclage modulo 16. Les instructions de branchement acceptent une adresse encodée 0..15, mais le programme doit effectivement contenir la destination. Un PC hors de la longueur chargée déclenche FAULT au prochain créneau valide.

OUT et OUT 1 adressent deux groupes de sept sorties ; ce ne sont pas deux banques de huit. O7 est dans OUT 1, alors qu'O7 est lu par IN. Pour une broche isolée, SET/OUTBIT/INBIT utilisent les indices absolus de la section 5.

Encodages invalides

0x30F, 0x3C0..0x3FF, les indices entrée 13..15 et sortie 14..15 sont réservés. Leur exécution lève FAULT et arrête RUN. Les 944 combinaisons canoniques de l'assembleur sont distinctes ; les 80 autres mots de 10 bits sont testés comme invalides. SIGNAL, RECV et OUTMSB ne font pas partie de cette ISA.

Synchronisation logicielle

EVENT est un drapeau d'un bit, pas un compteur ni une file. Des événements répétés avant consommation se confondent. Les affectations dues à une instruction interviennent après celles de l'accès hôte au même cycle ; ne pas utiliser des écritures concurrentes SET/CLEAR/AWAIT comme protocole de comptage sans acquittement logiciel.

Le PIO n'a pas d'interruption entrante qui préempte son programme. AWAIT/WAIT constituent des attentes coopératives. L'IRQ émise avertit l'hôte ; elle ne change pas le flot d'exécution du PIO.

15 Moteur temporel autonome

Le moteur reconna t une nouvelle trame apr s une dur e configurable de niveau bas sur DIN. Chaque front montant d clenche un  chantillonnage retard  de DIN : sa valeur   cet instant d termine le bit. La sortie est lanc e apr s un autre d lai, avec une largeur haute choisie selon ce bit ou selon le pr fixe de remplacement.

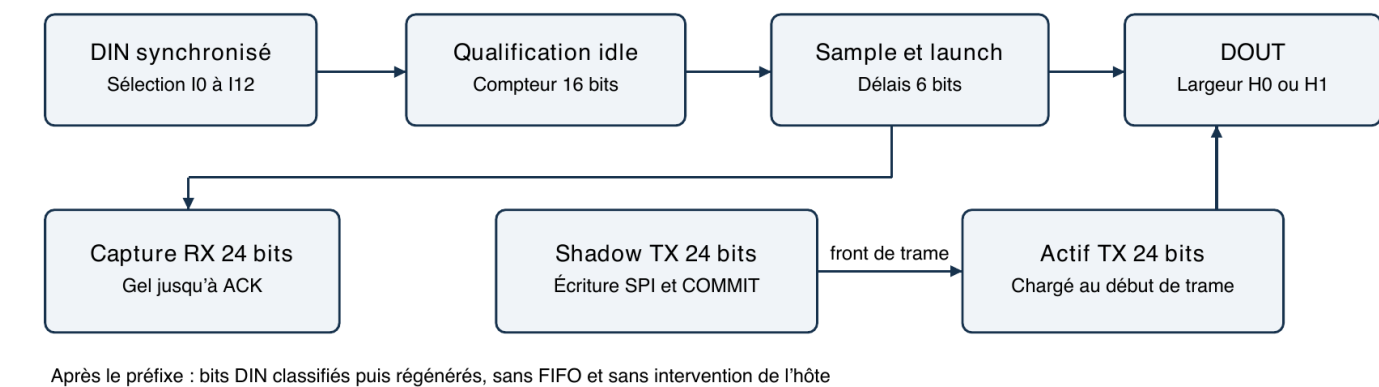


Figure 3 Chemin de donn es du moteur temporel

Mode ENABLE=1	OUTPUT	REPLACE	R�sultat
Capture seule	0	0	Acquisition du pr�fixe ; pas de r�servation DOUT
Relais r�g�n�r�	1	0	Pr�fixe et suite retransmis selon H0/H1
Remplacement et relais	1	1	Pr�fixe TX actif, puis bits DIN r�g�n�r�s

Le pr fixe re u est MSB en premier et align    droite dans RX24. En remplacement, les N bits bas du mot actif sont  mis du bit N 1 au bit 0. La suite de la trame n'est pas stock e : elle peut d passer 24 bits et se poursuit tant que l'entr e respecte les d lais.

La sortie n'est pas un court-circuit  lectrique DIN/DOUT. Les fronts sont retard s et les impulsions hautes r g n r es. Sans fronts DIN, le moteur ne g n re pas de trame autonome. Un compteur 24 bits  ventuel est calcul  par le LPC puis charg  comme donn e TX ; aucun incr menteur 24 bits d di  n'existe dans le silicium.

16 Registres du moteur temporel

Adr.	Nom	Accès	Reset	Champs et effet
60	TIMED_ID	R	5449	Identité de l'extension
61	TIMED_CTRL	R/W	0000	Modes [2:0] ; strobes ACK[8], COMMIT[9], CLEAR[10]
62	TIMED_PINS	R/W	0080	[3:0] DIN 0..12 ; [7:4] DOUT 0..13 ; [15:8]=0
63	IDLE_CYCLES	R/W	3A98	1..65535 cycles bas pour armer une trame ; défaut 15000
64	SAMPLE_LAUNCH	R/W	2019	[5:0] sample ; [13:8] launch ; $1 \leq \text{sample} < \text{launch} \leq 63$
65	HIGH_WIDTHS	R/W	2010	[5:0] H0 ; [13:8] H1 ; chaque valeur 1..63
66	PREFIX_BITS	R/W	0018	Longueur N de 1 à 24 ; autres bits nuls
67	TX_SHADOW_LO	R/W	0000*	Bits [15:0] du staging TX
68	TX_SHADOW_HI	R/W	0000*	Bits [7:0] = TX[23:16] ; bits [15:8] de W doivent être 0
69	RX_PREFIX_LO	R	0000*	Bits [15:0] reçus ; zéro si RX_VALID=0
6A	RX_PREFIX_HI	R	0000*	Bits [7:0] = RX[23:16], autres bits zéro ; zéro si invalide
6B	TIMED_STATUS	R	0000	Drapeaux détaillés section 17
6C	TIMED_VERSION	R	0118	Révision 0x01 ; capacité maximale 0x18 = 24 bits

* Valeur de lecture masquée : les banques de données ne sont pas physiquement remises à zéro. Chaque moitié TX devient lisible après une première écriture acceptée ; RX devient lisible uniquement après acquisition complète.

Conditions d'écriture

Les registres 0x62..0x66 ne se modifient que moteur désactivé. Les deux champs des registres 0x64 et 0x65 sont limités à six bits : les bits [7:6] et [15:14] doivent être nuls. Les écritures non conformes lèvent TIMED_HOST_ERROR et ne modifient pas le paramètre.

Les registres TX_SHADOW peuvent être écrits pendant une trame, mais pas quand COMMIT_PENDING vaut 1. Le premier COMMIT exige que les deux moitiés aient été initialisées. Une fois initialisées, elles le restent jusqu'au reset ; pour éviter une valeur composée involontaire, le pilote réécrit et relit systématiquement les deux moitiés.

Les lectures RX sont non destructives : il faut un ACK explicite pour libérer la donnée. Les adresses 0x6D..0x6F se lisent à zéro ; toute écriture non implémentée de la page 0x60 déclenche TIMED_HOST_ERROR, y compris une écriture sur un registre R.

17 Contr le et  tat du moteur temporel

Bit de 0x61	Nom	Fonction
0	ENABLE	Active le moteur ; z�ro abandonne la trame
1	OUTPUT_ENABLE	Autorise DOUT et sa r�servation si ENABLE=1
2	REPLACE_PREFIX	�met le mot actif pendant le pr�fixe
8	ACK_RX	Strobe d'acquittement RX_VALID
9	COMMIT	Strobe : verrouille shadow en attente de prochaine trame
10	CLEAR_ERRORS	Strobe : efface les trois erreurs temporelles
Autres	R�serv�s	�crire z�ro ; les strobes se lisent � z�ro

Une  criture de contr le remplace toujours les bits de mode [2:0]. Pour acquitter tout en conservant le mode remplacement/relais,  crire 0x0107, pas 0x0100. Pour effacer les erreurs dans ce mode,  crire 0x0407.

Bit de 0x6B	Nom	Signification
0	ARMED	Dur�e de bas qualifi�e ; attend le premier front
1	IN_FRAME	Une trame a commenc� et n'a pas encore �t� termin�e/abandonn�e
2	RX_VALID	Pr�fixe complet conserv� jusqu'� ACK
3	COMMIT_PENDING	Shadow verrouill� ; transfert vers actif au prochain d�part de trame
4	ACTIVE_TX_VALID	Un mot TX actif a �t� charg�
5	HOST_ERROR	�criture ou combinaison de contr�le interdite
6	RX_OVERRUN	D�but de nouvelle trame alors que RX pr�c�dent non acquitt�
7	TIMING_ERROR	Front trop rapproch� ou collision avec une impulsion DOUT
15:8	R�serv�s	Z�ro

IRQ temporelle = RX_VALID OR HOST_ERROR OR RX_OVERRUN OR TIMING_ERROR, y compris moteur d sactiv . Aucun masque d'IRQ temporelle n'existe.

Commandes refus es

Un changement des bits OUTPUT/REPLACE tout en gardant ENABLE=1, un conflit de broche, un COMMIT d j  pending ou non initialis , ou l'activation du remplacement sans mot actif ni commit admissible, sont rejet s. D sactiver d'abord pour changer de mode. CLEAR_ERRORS ne corrige pas une commande invalide et ne supprime pas RX_VALID.

18 D lais impulsions et fronti res de trame

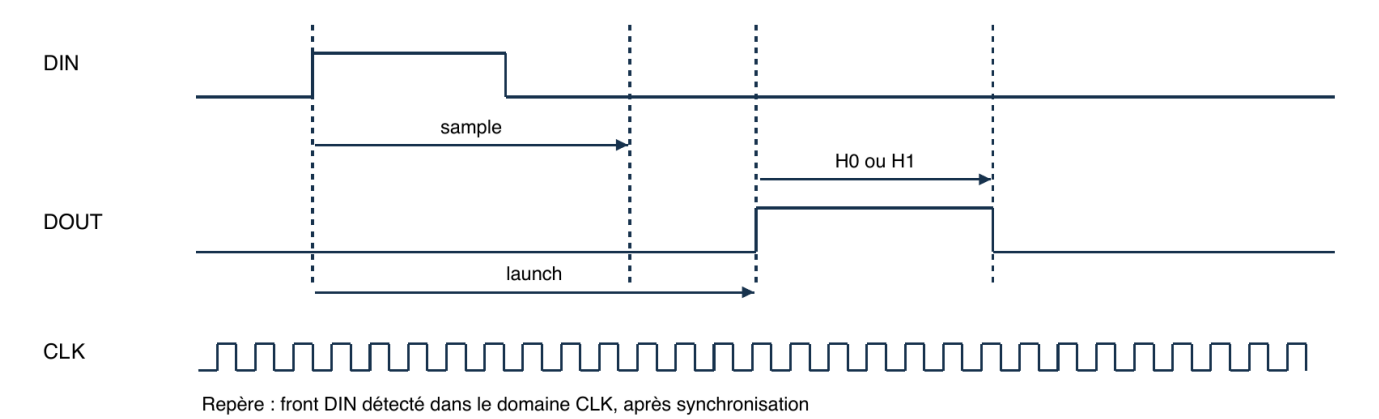


Figure 4 Chronogramme de principe ; non   l chelle

Au front interne de d part, un compteur d ge est lanc . DIN est classifi  apr s SAMPLE cycles ; DOUT monte apr s LAUNCH cycles. Sa dur e haute est H0 ou H1 cycles. Le prochain front DIN fixe la p riode suivante : aucun compteur autonome de p riode de bit ne cadence la trame.

Param�tre	Plage de programmation	Dur�e � 50 MHz
IDLE	1..65535 cycles	20 ns..1,3107 ms
SAMPLE	1..62, strictement inf�rieur � LAUNCH	20 ns..1,24 �s
LAUNCH	2..63 cycles	40 ns..1,26 �s
H0, H1	1..63 cycles chacun	20 ns..1,26 �s

Une programmation dans ces bornes n est pas n cessairement une forme d onde valide. Le front suivant ne doit pas arriver pendant l chantillonnage en cours, ni une nouvelle sortie se lancer tant que la pr c dente est haute. Par lecture du RTL, imposer un intervalle interne entre d parts strictement sup rieur   max(LAUNCH, H0, H1). Ajouter la marge de synchronisation et les contraintes de la carte ; ce n est pas une fr quence maximale caract ris e.

Une erreur temporelle arr te la trame et force DOUT bas ; une nouvelle dur e basse qualifi e est n cessaire pour repartir. La classification ne v rifie pas toutes les tol rances d un protocole : des impulsions class es 0/1 peuvent rester hors sp cification du r cepteur externe.

RX est gel  tant que RX_VALID=1. Si une nouvelle trame commence avant ACK, RX_OVERRUN est pos  et le nouveau pr fixe n est pas captur  ; le relais peut continuer. Un ACK en milieu de cette trame ne r active pas sa capture. Une trame incompl te ne publie pas un mot partiel.

19 Int gration avec un LPC546xx

Liaisons logiques : adaptation  lectrique et brochage carte   d finir

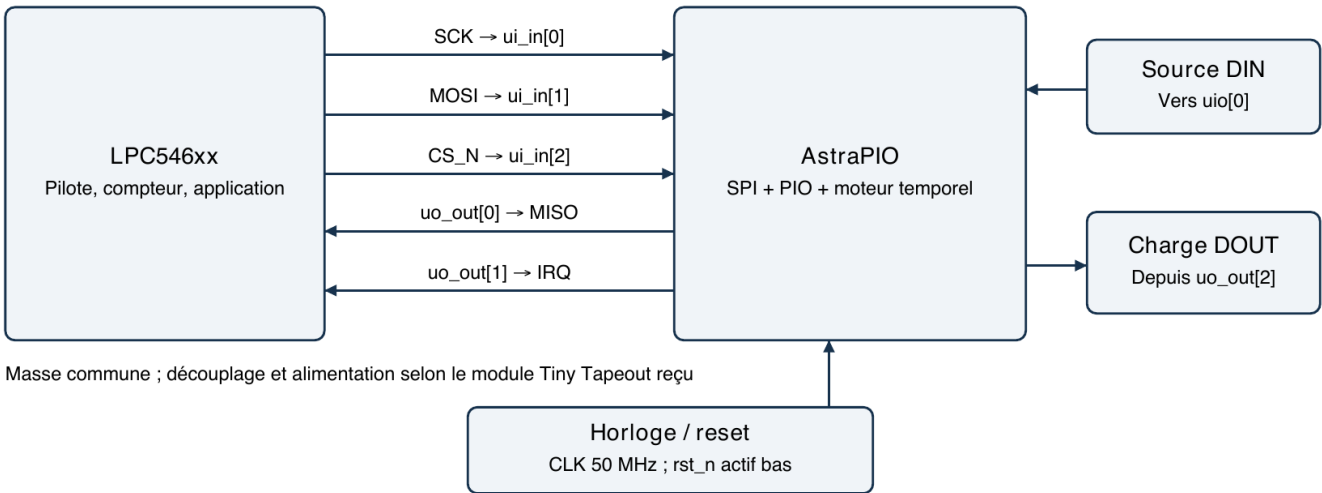


Figure 5 Sch ma de principe, sans num ro de broche de bo tier ni tension d'interface pr sum e

R partition des responsabilit s

LPC546xx	AstraPIO
Configure et v�rifie les param�tres au d�marrage	�chantillonne DIN et cadence DOUT dans le domaine CLK
Calcule la prochaine valeur 24 bits � la demande	Conserve une valeur active et une valeur en attente
Lit le pr�fixe re�u et l'acquitte apr�s traitement	G�le RX24 et l�ve IRQ jusqu'� acquittement
Traite erreurs, d�passements et besoins de l'application	Ex�cute le programme PIO ind�pendamment du moteur temporel

Le transfert SPI ne doit pas  tre ex cut  dans une interruption qui pr empte un autre transfert sur le m me p riph rique. Une t che propri taire du pilote peut traiter IRQ et les demandes de mise   jour. Un transfert DMA est possible, mais sa fin logique inclut le dernier front sur le fil et le temps de d s lection CS.

Cycle de vie d'une mise   jour

 crire les deux moiti s shadow → relire → COMMIT → attendre COMMIT_PENDING=0. Le mot actif ne change qu'au d but d'une nouvelle trame qualifi e. Une demande re ue au voisinage exact de ce d but ne garantit pas sa prise en compte dans cette trame : confirmer le commit suffisamment avant le premier front pour imposer une fronti re pr cise.

La r f rence LPC546xx exacte, ses broches SPI, sa configuration SDK, les niveaux  lectriques et l' ventuel translateur restent   choisir. Ce sch ma n'est pas un sch ma  lectrique pr t   fabriquer. MISO ne se lib re pas   CS_N=1 : r server une entr e MISO ou ajouter une isolation adapt e.

21 Exemples de programmes PIO

Ces programmes utilisent l'interpréteur général, pas le moteur temporel. Ils sont assemblés pour ABI 5 et ont été exercés par les tests de broches. Les débits supposent CLK = 50 MHz et des données disponibles dans la FIFO TX.

Émission UART 8N1 sur uio[0] — 14 instructions

```
LDI 1          # direction sortie pour uio[0]
DIR
SET 0,1        # repos haut ; masque de propriété = 0x0001
byte: PULL     # attend un octet TX
LDX 8
SET 0,0        # start
DELAY 106
bit: OUT       # ACC[0] vers uio[0], seul bit possédé
SHR
DELAY 104
DJNZ bit
SET 0,1        # stop
DELAY 106
JMP byte
```

La cellule de bit dure 108 créniaux, soit 8,640 µs : débit calculé de 115 740,7 bauds, environ +0,47 % par rapport à 115 200. PULL peut prolonger le repos entre octets. Pas de réception UART, de parité ou de contrôle de flux matériel dans cet exemple.

Émission série type SPI sur uio[0:1] — 9 instructions

```
LDI 3          # MOSI=uio[0], SCK=uio[1]
DIR            # masque de propriété = 0x0003
byte: PULL
LDX 8
bit: OUTBIT 0  # MSB en premier, puis décalage à gauche
SET 1,1
SET 1,0
DJNZ bit
JMP byte
```

À l'intérieur d'un octet, SCK reste haut 80 ns et bas 240 ns, soit 3,125 Mbit/s calculés avec un rapport cyclique de 25 %. Une pause supplémentaire existe entre octets et lorsque TX est vide. Le test vérifie les bits et la largeur haute. Cet exemple ne gère ni CS ni MISO ; ce n'est pas un contrôleur SPI complet.

Autres usages : périmètre raisonnable

Séquences de contrôle, trains d'impulsions courts et protocoles série simples sont programmables dans les limites de 16 instructions, ACC8 et FIFO de deux octets. I²C exige une direction de sortie adaptée, des résistances de rappel et la gestion des acquittements/étirements ; aucune pile I²C complète n'est livrée. Un contrôleur VGA complet ou un décodeur ADS-B/Mode S ne fait pas partie des fonctions qualifiées.

22 Pilote portable C et interface hôte

Les fichiers firmware/pio_host.[ch] et firmware/pio_timed.[ch] fournissent un pilote sans dépendance au SDK NXP. Le transport propre à la carte réalise exactement une transaction SPI par appel. Les arguments context doivent être zéro ; le masque de RUN est 0 ou 1.

```
typedef int (*pio_transfer_fn)(void *ctx,
    const uint8_t tx[4], uint8_t rx[4]);
typedef struct {
    pio_transfer_fn transfer;
    void *ctx;
} pio_device;
```

Fonction / famille	Responsabilité
pio_probe / pio_timed_probe	Contrôle les identités et versions attendues
pio_load / pio_set_entry	Charge/relit le programme ; positionne l'entrée, PIO arrêté
pio_set_mask / pio_run / pio_stop	Propriété des sorties et marche/arrêt du contexte
pio_push / pio_push_many / pio_pop	FIFO octets ; push_many utilise jusqu'à deux crédits TX
pio_timed_configure	Désactive, configure et relit ; ne démarre pas le moteur
pio_timed_stage	Vérifie, écrit et relit shadow24, puis commit atomique
pio_timed_receive	Lit RX24 de façon stable, sans acquittement automatique
pio_timed_control	Modes et ACK/CLEAR ; COMMIT réservé à stage()

Code	Valeur	Traitement attendu
PIO_OK	0	Opération terminée selon le contrat de la fonction
PIO_EINVAL	-1	Corriger argument ou configuration
PIO_EIO	-2	Effet du transfert potentiellement incertain ; resynchroniser
PIO_EABI	-3	Identité/version incompatible ; ne pas démarrer
PIO_EVERIFY	-4	Écart de relecture ; garder l'application à l'arrêt
PIO_EEMPTY	-5	Pas de donnée disponible ; attendre ou sonder
PIO_EFULL	-6	Plus de crédit TX ou commit en attente ; différer
PIO_EFAULT	-7	Lire les états matériels, corriger puis réarmer

Séquence applicative recommandée

```
initialiser le transport et assurer le reset du module
vérifier PIO_ID, ABI et TIMED_ID / TIMED_VERSION
configurer les sorties et le moteur, puis vérifier les erreurs
stage(compteur_initial) ; control(ENABLE | OUTPUT | REPLACE)
sur IRQ : lire status, traiter RX, puis ACK si donnée acceptée
sur demande : stage(compteur_suivant), avancer seulement si succès
```

PIO_OK de pio_write indique une réussite du callback, pas un acquittement électrique du composant : SPI ne comporte pas d'ACK. Pour les opérations critiques, vérifier les états/relectures. pio_load ne purge pas à lui seul HOST_ERROR ; le traiter avant le démarrage. Après une erreur EIO, ne pas réessayer aveuglément une lecture RX destructrice, un push ou un commit.

23 Erreurs, récupération et limites

Événement	Effet matériel	Réaction hôte
Instruction invalide / PC hors programme	FAULT ; PIO arrêté ; IRQ	Arrêter, diagnostiquer le programme, RESTART puis recharger si nécessaire
Écriture PIO interdite / TX plein	HOST_ERROR mémorisé ; écriture refusée	Corriger la cause, écrire 1 à 0x06 ; RESTART ne suffit pas
FIFO RX pleine	PUSH se bloque ; pas de perte automatique	Vider RX avec lectures complètes de 0x15
FIFO TX vide	PULL se bloque, sorties conservées	Remplir TX ; aucun délai maximum matériel
Écriture timed invalide / conflit de broche	TIMED_HOST_ERROR ; paramètre inchangé	Désactiver pour reconfigurer, conserver les modes sur ACK/CLEAR
Préfixe RX non acquitté à la trame suivante	RX_OVERRUN ; nouvelle capture ignorée	Traiter l'ancienne donnée, ACK puis CLEAR ; attendre une nouvelle trame
Fronts ou impulsions en collision	TIMING_ERROR ; trame abandonnée, DOUT bas	Corriger paramètres/entrée ; CLEAR ; attendre un bas qualifié
COMMIT_PENDING reste à 1	Pas encore de début de trame qualifié	Attendre DIN ; aucune seconde valeur shadow ne peut être écrite
Erreur de transport incertaine	Effet inconnu à partir du point d'erreur	Relire états non destructifs ; réinitialiser si la cohérence ne peut être établie

Ce qu'un arrêt ne remet pas à zéro

RUN=0 conserve les sorties du PIO, la direction et son masque. RESTART efface l'état d'exécution et les FIFO mais ne réinitialise pas le moteur temporel. Désactiver le moteur temporel abandonne la trame, mais conserve RX_VALID, les erreurs, le mot actif et le commit en attente. Un commit en attente ne peut pas être annulé par une simple désactivation : il faut son transfert au prochain départ qualifié ou un reset global.

Limites à intégrer dès la conception

- Pas de RAM générale, de macro SRAM, de second contexte PIO, de pile, de multiplication ou de tampon de trame complète.
- Pas de watchdog protocolaire, de CRC SPI, de reprise automatique ni de masque global de toutes les interruptions.
- Le relais temporel dépend des fronts DIN ; il ne crée pas une trame WS2812 à partir du seul mot TX.
- Un arrêt d'horloge empêche la progression des machines synchrones. Prévoir au niveau système un moyen sûr de reset et de remise en état des sorties.

L'utilisation dans une fonction de sécurité exige une analyse et une qualification spécifiques absentes de ce projet. Aucun comportement de sûreté système ni niveau de certification n'est revendiqué.

24 Qualification, tra abilit  et r f rences

Validation conserv�e	R�sultat / port�e
Simulation de broches	25 sc�narios RTL et 25 sur netlist rout�e, sans SDF
Diff�rentiel ABI 4 → ABI 5	1988 traces : 944 op�rations × 2 �tats + 100 programmes al�atoires
Unit�s et logiciel	FIFO, m�moire 10/16 bits, assembleur, protocole, pilotes C avec sanitizers
Placement/routage et prechecks	Z�ro violation DRC Magic/KLayout, LVS, XOR, antenne ; 10 prechecks
Application WS2812	12/12 tests par mode ; sc�nario compteur : 1440 bits DOUT par mode
Audit STA compl�mentaire	3 coins, early 0,95 / late 1,05 ; parasites nominaux, CLK 20 ns
Soumission	PR149 fusionn�e le 19/09/2026 � 15:14:01 UTC ; contr�les centraux r�ussis

Marges hold de l’audit : fast 1,32 V/−40  C : +0,004872 ns ; typical 1,20 V/25  C : +0,140331 ns ; slow 1,08 V/125  C : +0,375741 ns. La marge fast est  troite. Il ne s’agit pas d’une caract risation silicium ni d’un domaine garanti de tension/temp rature.

Source : 1b1c91183a4a9a5ea3516699845336175ffe6d96
PDK : c4b8b4e5e7a05f375cca3815d51b3a37721fbf5c ; LibreLane 3.0.5
SHA-256 netlist finale :
6d2dbfd333bb89544cf296e9ff408e8870b7151984e7962b098833fd15e3e948

R f rences de d finition et preuves

- [R1] D finition fonctionnelle : src/project.v, pio_single_core.v, pio_spi.v, pio_timed.v, pio_program.v et pio_fifo.v du commit ci-dessus.
<https://github.com/fvannel/AstraPIO/tree/1b1c91183a4a9a5ea3516699845336175ffe6d96/src>
- [R2] Pilotes, assembleur et tests : firmware/, tools/pioasm.py, test/ ; synth se locale docs/validation-finale.md et test/ws2812/README.md. Les commentaires historiques ne remplacent pas les identit s ABI de ce document.
- [R3] Build physique officiel :
<https://github.com/fvannel/AstraPIO/actions/runs/35442726541>
- [R4] Audit temporel compl mentaire :
<https://github.com/fvannel/AstraPIO/actions/runs/35448283122>
- [R5] Soumission et contr le central :
<https://github.com/TinyTapeout/tinytapeout-ihp-26b/pull/149>
<https://github.com/TinyTapeout/tinytapeout-ihp-26b/actions/runs/35450180122>
- [R6] R f rence locale immuable : release/manifest.json, release/validation/ et tools/check_final_release.py. Les 50 empreintes prot g es ont  t  v rifi es lors du nettoyage du 20/09/2026.
- [R7] Documentation Tiny Tapeout consult e le 20/09/2026 :
<https://tinytapeout.com/specs/gpio/>
<https://tinytapeout.com/specs/clock/>
<https://tinytapeout.com/specs/pinouts/>
Les limites des pages GPIO/Clock concernent SKY130 et ne sont pas transpos es   ce design IHP.
- Historique documentaire : r vision 1.0, premi re datasheet consolid e pour ABI 5. Aucun changement de circuit associ . Fabrication, bring-up carte, simulation SDF et caract risation  lectrique restent hors des r sultats pr sent s.